

Here is the fully updated Architecture Design Document incorporating the shift from shared-memory polling to the highly optimized, $O(1)$ **Direct Messaging** approach using `onvm_nflib_send_msg_to_nf()`, as well as the correction regarding the DPU Agent's process type.

1. Executive Summary

The ultimate goal of this architecture is to offload 5G User Plane Function (UPF) data-plane processing—specifically Packet Detection Rules (PDRs), Forwarding Action Rules (FARs) including GTP-U encap/decap, and QoS Enforcement Rules (QERs)—directly into the hardware silicon of an NVIDIA BlueField-3 DPU.

To overcome DPDK Multi-Process limitations while maintaining a pure software core, we adopt a **Split-Agent Architecture** modeled after the *DOCA Secure Channel* reference application. The host maintains pure 5G logic, a Host Agent acts as an IPC bridge via ONVM direct messaging, and a standalone DPU Agent programs the hardware.

2. Component Architecture

2.1 Host Node (x86 Server)

The Host environment utilizes the existing OpenNetVM (ONVM) framework.

- **onvm_mgr (DPDK Primary Process):** Owns the Virtual/Scalable Functions (VFs/SFs) exposed by the DPU. It polls traffic and schedules it to Network Functions (NFs) via lockless software rings.
- **upf_u (DPDK Secondary Process):** The Data Plane NF. Acts as the **Software Fallback**. Processes any N3/N6 packets that miss the hardware rules, handling complex buffering (e.g., paging) or unsupported packets.
- **upf_c (DPDK Secondary Process):** The Control Plane NF. Terminates PFCP. It writes new rules to the DPDK shared memory for the software fallback, and actively pushes hardware-specific offload instructions to the Host Agent. **No DOCA code is compiled into upf_c.**
- **Host Offload Agent (DPDK Secondary Process):** A newly introduced ONVM NF. It registers a message handler callback (`msg_handler`) to receive specific hardware offload structs

directly from `upf_c` via ONVM lockless rings, bridging the data to the DPU via the DOCA Comm Channel.

2.2 DPU Node (BlueField ARM Cores)

- **DPU Offload Agent (Standalone DOCA Application):** A lightweight C application running natively on the DPU ARM OS. It naturally initializes as the primary owner of the physical HW devices/queues (no explicit `--proc-type` flag needed). It initializes DOCA Flow in **Switch Mode** ("`switch,hws`"), manages physical/representor ports, receives IPC messages from the Host Agent, and programs the e-switch silicon.
-

3. Control Plane Workflow (Rule Provisioning)

We utilize an $O(1)$ direct messaging approach to decouple the speed of hardware rule insertion from the latency of bulk software classifier rebuilds. How a PFCP message turns into a hardware silicon rule:

1. **PFCP Arrival:** `upf_c` receives a Session Establishment/Modification request from the SMF and parses the PDRs, FARs, and QERs.
2. **Software State Update (For Fallback):** `upf_c` updates the global `5gc_classifiers` and session tables, pushing the new snapshot into the `UPF_CLS_CTRL_SLOT` memzone. It sends the standard ONVM event `UpfSendEvt(UPF_U_SERVICE_ID, EVT_CLS_GC_REQ, ver);` so `upf_u` can handle any packets that miss the hardware.
3. **Hardware Payload Construction:** `upf_c` extracts only the hardware-relevant Match Criteria (e.g., TEID, UE IP) and Actions (e.g., Encap/Decap, Drop, MBR). It allocates a compact `hw_offload_msg` C-struct (representing a rule "Diff") in DPDK shared hugepage memory using `rte_malloc` and populates these fields.
4. **Direct Message Passing:** `upf_c` sends this struct directly to the Host Offload Agent using ONVM's lockless inter-process messaging API:
`onvm_nflib_send_msg_to_nf(HOST_AGENT_ID, msg).`
5. **Host Agent Reception & Comm Channel TX:** The message triggers the Host Agent's asynchronous `msg_handler` callback. The Host Agent casts the payload, transmits the exact `hw_offload_msg` struct across the PCIe bus using the `doca_comch_client` API, and then

frees the hugepage memory (`rte_free`).

6. **Hardware Insertion:** The DPU Agent receives the compact payload via `doca_comch_server`. It maps the struct fields directly into `doca_flow_match` and `doca_flow_actions` primitives, calling `doca_flow_pipe_add_entry()` to install the specific rule into the NIC silicon instantly.

4. Data Plane Workflow & Pipeline Design

The DPU Agent configures DOCA Flow using a **Control Pipe Hierarchy** to support mixed exact/wildcard matching and routing.

- **Root Control Pipe:** Matches incoming traffic type (e.g., UDP port 2152 for GTP-U) and routes to specific Basic Pipes.
- **Basic Pipes (The Templates):** Pre-configured masks defining rule types (e.g., exact match on `src_ip`, wildcard on `dst_ip`).
- **Fast Path (Hardware Hit):** Packets hitting a PDR execute the FAR in silicon. Hardware performs GTP-U decapsulation (uplink), GTP-U encapsulation (downlink), MAC modifications, and routes out the physical N6/N3 ports. *Zero Host CPU cost.*
- **Slow Path (Hardware Miss):** If a packet misses all pipes, the final `fwd_miss` rule routes the packet to the **Host Representor port**. The packet travels over PCIe to `onvm_mgr` and into `upf_u` for software processing.

5. QoS & Traffic Shaping Strategy (Two-Phase Rollout)

Handling 5G QoS requires careful management of the SmartNIC's memory and compute architectures. We adopt a two-phase rollout to balance immediate engineering pragmatism with advanced academic research goals.

Phase 1: Hybrid QoS (HW Policing + SW Shaping)

- **Hardware Policing (MBR Enforcement):** For flows requiring basic rate-limiting/dropping (Maximum Bit Rate), the DPU Agent natively attaches a `DOCA_FLOW_RESOURCE_METER`

(trTCM) to the DOCA Flow rule. Excess packets are dropped in silicon instantly.

- **Software Shaping (GBR / Complex Pacing):** For flows requiring deep buffering or hierarchical shaping, the DPU Agent is instructed *not* to offload forwarding (using `HW_ACTION_PUNT` in the message payload). The rule directs these packets to the Host Representer. `upf_u` utilizes DPDK's x86 hugepage memory and `rte_sched` to buffer and shape traffic before transmitting.

Phase 2: Ultimate Research Goal (DPA Bare-Metal Shaping)

To achieve mathematically precise, jitter-free Hierarchical QoS, shaping will be migrated entirely off the Host OS and onto the BlueField's Data Path Accelerator (DPA) RISC-V cores.

1. **DPA Buffer Pool:** The DPU Arm CPU allocates a large chunk of onboard DDR RAM and wraps it in a `doca_mmap`.
2. **Traffic Intercept:** DOCA Flow rules forward QoS-bound traffic to the DPA Rx Queue instead of the Host. Packets are DMA'd into the `doca_mmap` space.
3. **FlexIO Execution:** A custom C application written using the **FlexIO SDK** runs bare-metal on the DPA cores. It holds the packet pointers, calculates real-time token buckets, and issues exact-microsecond packet transmission (Tx) instructions, completely bypassing Linux OS jitter.

6. Development Directives & Constraints

1. **Keep `upf_c` Clean:** The `upf_c` and `upf_u` applications must remain 100% hardware-agnostic. They only interact with standard DPDK/ONVM memory and events. Do not include DOCA headers in them.
2. **Struct Memory Management:** When `upf_c` creates a hardware offload message, it must use `rte_malloc()` so the memory is visible to the Host Agent. The Host Agent is responsible for calling `rte_free()` after transmission.
3. **Endianness Awareness:** `upf_c` and `upf_u` operate in Little Endian. Network rules require Big Endian. The Host Agent (or `upf_c` prior to sending) must enforce `rte_cpu_to_be_32()` when converting IPs and Ports for the `hw_offload_msg` payload.

4. **Process Types:**

- Host Agent = `--proc-type=secondary` (Inherits hugepages to read ONVM rings).
- DPU Agent = Standalone application (Defaults to Primary, owns physical HW).

5. **Hardware Steering (hws):** Ensure the DPU Agent calls `doca_flow_cfg_set_mode_args(cfg, "switch,hws")`. Switch mode is strictly required; VNF mode will fail to route exception traffic to the Host VFs.

1.